

REPÚBLICA BOLIVARIANA DE VENEZUELA

MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA

UNIVERSIDAD NACIONAL EXPERIMENTAL

DE TELECOMUNICACIONES E INFORMÁTICA (UNETI)

PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

DOCUMENTACIÓN TÉCNICA: DESARROLLO DE FORMULARIOS REACTIVOS Y ARQUITECTURA STANDALONE EN ANGULAR

Evaluación de la Sesión Didáctica 2

Profesor Académico

Carlos Alberto Márquez Guillen

Autor

Juan Henríquez

C.I: 27.913.162

Caracas, Junio de 2026

AGRADECIMIENTOS

A mi familia, por ser el soporte incondicional y la principal fuente de motivación a lo largo de mi formación universitaria, brindándome el respaldo necesario para mantener el enfoque frente a cada nuevo desafío tecnológico.

Al Profesor Carlos Alberto Márquez Guillen, por su invaluable orientación, nivel de exigencia y resiliencia frente a las adversidades. Su visión pedagógica en la cátedra de Programación III ha sido el catalizador para comprender la verdadera ingeniería detrás de las interfaces y dar el salto profesional hacia arquitecturas de software reactivas.

A mis colegas desarrolladores y a la vibrante comunidad de aprendizaje de la UNETI, por transformar esta etapa académica en un verdadero hub de innovación colaborativa.

La solidez técnica reflejada en este portafolio es el resultado directo de las incontables horas de debugging compartido, los debates constructivos sobre arquitectura de software y la camaradería inquebrantable que define a nuestra cohorte.

Finalmente, al esfuerzo personal invertido en la reingeniería de este proyecto, demostrando que la disciplina y el estudio autodidacta son las herramientas definitivas para dominar el desarrollo web moderno.

UNIVERSIDAD NACIONAL EXPERIMENTAL**PARA LAS TELECOMUNICACIONES E INFORMÁTICA (UNETI)****PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA**

Juan Henriquez

C.I: 27.913.162

Profesor: Carlos Márquez

Fecha: Junio, 2026

RESUMEN

El presente informe técnico detalla la evolución arquitectónica y funcional del proyecto prog3-architecture-logic, desarrollado bajo los lineamientos de la Sesión Didáctica 2 (SD2) de la Unidad Curricular Programación III. Se ejecutó una refactorización integral que abandona el paradigma tradicional de módulos en Angular para adoptar una Arquitectura Standalone, optimizando significativamente el rendimiento, el overhead y la mantenibilidad del código. La interfaz inicial fue transformada en un "Laboratorio Interactivo de Consultoría" dinámico, logrando esto mediante la implementación de Formularios Reactivos con validación atómica estricta y un sistema de gestión de estado en memoria local estructurado bajo el patrón Micro-CRUD. Complementado con un diseño visual basado en el estándar Bento-Grid y un sistema de retroalimentación asíncrona a través de componentes nativos de Ionic, este proyecto demuestra el dominio práctico en la construcción de aplicaciones web híbridas escalables, altamente interactivas y orientadas a una experiencia de usuario (UX) moderna.

PALABRAS CLAVE: Angular Standalone, Micro-CRUD, Reactive Forms, Bento-Grid, Ionic 8, Arquitectura de Software.

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVOS.....	5
1.1 Objetivos Específicos de la Sesión Didáctica (SD2).....	6
1.2. Propósito de la Reingeniería.....	7
1.3. El Concepto: "Laboratorio Interactivo de Consultoría"	8
2. MIGRACIÓN ARQUITECTÓNICA: HACIA STANDALONE COMPONENTS.....	10
2.1. Eliminación del Monolito NgModule.....	10
2.2. Configuración de Arranque (bootstrapApplication y app.config.ts).....	11
2.3. Impacto en el Rendimiento y Modularidad.....	12
3. INGENIERÍA DE FORMULARIOS REACTIVOS.....	14
3.1. Implementación de ReactiveFormsModule.....	14
3.2. Validación Atómica y Reglas de Negocio (Required, MinLength, Email).....	15
3.3. Reactividad en la Interfaz (Desbloqueo condicional del botón de envío).....	17
4. GESTIÓN DE ESTADO Y MICRO-CRUD.....	19
4.1. Arquitectura de Inyección de Dependencias (ContactService).....	19
4.2. Operaciones en Memoria Local (Create, Read, Delete).....	20
4.3. Renderizado Dinámico de la Bitácora de Sesión.....	21
5. DISEÑO UI/UX Y FEEDBACK MULTIMEDIA.....	22
5.1. Estructura visual: Patrón Bento-Grid.....	22
5.2. Feedback Nativo Integrado (ToastController de Ionic).....	23
5.3. Micro-interacciones (Badges de estado pulsantes).....	24
6. CONCLUSIONES.....	25
REFERENCIAS BIBLIOGRÁFICAS.....	26

1. INTRODUCCIÓN Y OBJETIVOS

La presente Sesión Didáctica 2 (SD2) de la asignatura Programación III tiene como objetivo fundamental la evolución arquitectónica y funcional del proyecto web híbrido. Esta fase marca la transición de una interfaz puramente visual y estática hacia una aplicación robusta, interactiva y con gestión de estado en memoria.

Para garantizar la trazabilidad y disponibilidad del proyecto, todos los cambios arquitectónicos y la reestructuración de archivos han sido debidamente integrados y documentados en el repositorio oficial de GitHub:

<https://github.com/KainSutcliff1607/prog3-architecture-logic>.

Asimismo, la versión final de esta reingeniería cuenta con despliegue continuo y se encuentra accesible en producción a través de la plataforma Vercel en el siguiente enlace:

<https://prog3-architecture-logic.vercel.app/>, asegurando que los evaluadores puedan interactuar con la versión más reciente en tiempo real.

1.1 Objetivos Específicos de la Sesión Didáctica (SD2)

En alineación con los requerimientos académicos establecidos para esta evaluación, el desarrollo se centró en alcanzar los siguientes objetivos técnicos:

Migración a Componentes Standalone: Eliminar la dependencia de la arquitectura basada en NgModule para reducir el overhead y optimizar el rendimiento general de la aplicación.

Implementación de Formularios Reactivos (ReactiveFormsModule): Desarrollar un módulo de contacto que implemente validaciones atómicas estrictas y reactividad en la interfaz de usuario.

Gestión de Estado (Micro-CRUD): Integrar un servicio mediante inyección de dependencias para el manejo dinámico de los datos en sesión (creación, lectura y eliminación de registros), sin persistencia externa.

Optimización de Activos y UI/UX: Aplicar estrategias de rendimiento multimedia e integrar retroalimentación visual asíncrona mediante el ToastController de Ionic.

1.2. Propósito de la Reingeniería

El propósito central de esta actualización técnica es modernizar el núcleo de la aplicación, abandonando prácticas legacy para adoptar estándares actuales de desarrollo con Angular e Ionic. La reingeniería se ejecutó sobre tres pilares técnicos:

Migración a Arquitectura Standalone:

Se eliminó la dependencia estructural de los módulos tradicionales (NgModule), transformando los componentes de la aplicación a entidades independientes. Esto reduce drásticamente el overhead de la aplicación y mejora los tiempos de carga (Lazy Loading implícito).

Implementación de Formularios Reactivos:

Se reemplazó el enfoque impulsado por plantillas (Template-driven) por el módulo ReactiveFormsModule. Esto permite trasladar toda la lógica de validación (reglas atómicas como longitud mínima, requerimientos y formato de correos) directamente a la capa de TypeScript, garantizando la integridad de los datos antes de cualquier procesamiento.

Inyección de Dependencias y Estado:

Se diseñó un servicio desacoplado de la vista para manejar la lógica de negocio, cumpliendo con los principios de responsabilidad única (SOLID) exigidos en la formación académica.

1.3. El Concepto: "Laboratorio Interactivo de Consultoría"

Para superar el requerimiento básico de un "formulario de contacto" y dotar al proyecto de un alto valor agregado, se conceptualizó e implementó un **"Laboratorio Interactivo de Consultoría"**. Este enfoque transforma la experiencia del usuario, pasando de un simple envío de datos a un entorno de soporte técnico dinámico.

Las características principales de este concepto incluyen:

Arquitectura Visual (Patrón Bento-Grid):

La interfaz fue rediseñada dividiendo el espacio en dos áreas de trabajo simultáneas y responsivas: el "Cerebro de Operaciones" (donde se estructuran las consultas mediante selectores de categoría) y la "Bitácora de Sesión" (donde se reflejan los resultados).

Sistema Micro-CRUD en Memoria:

Se integró un motor de gestión de estado local que permite al usuario interactuar de manera completa con sus datos temporales. El sistema soporta la creación de *tickets* de soporte, su lectura instantánea en la interfaz y la eliminación de registros específicos.

Feedback Inmersivo e Inmediato:

Para consolidar la experiencia de usuario (UX), cada acción de escritura, creación o eliminación está respaldada por confirmaciones nativas utilizando el ToastController de Ionic, acompañado de indicadores visuales (badges de estado) que responden en tiempo real.

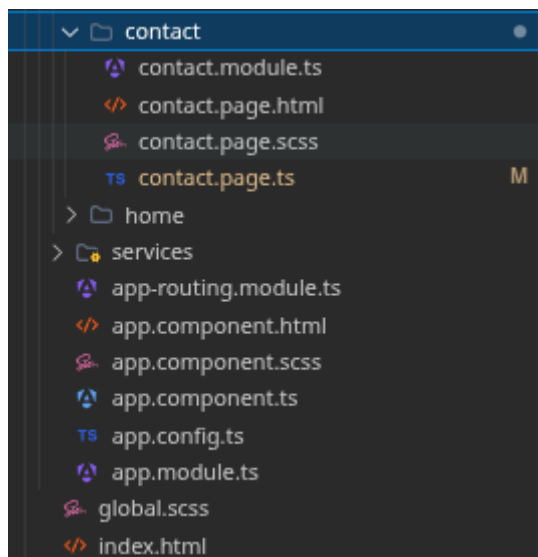
2. MIGRACIÓN ARQUITECTÓNICA: HACIA STANDALONE COMPONENTS

La actualización arquitectónica más significativa de esta Sesión Didáctica fue la transición hacia los Componentes Independientes (Standalone Components). Este paradigma moderno redefine la forma en que se estructuran las aplicaciones web híbridas, promoviendo un ecosistema de desarrollo más ligero y eficiente.

2.1. Eliminación del Monolito NgModule

En iteraciones anteriores del framework, era estrictamente necesario acoplar las vistas y directivas dentro de contenedores lógicos globales o específicos (@NgModule). Para cumplir con los requerimientos de la SD2, se ejecutó la eliminación sistemática de estos archivos intermediarios, destacando la remoción del `app.module.ts` y del archivo [contact.module.ts](#).

Al refactorizar el componente principal y el módulo de contacto incorporando el metadato `standalone: true`, la arquitectura transita de un modelo centralizado a uno distribuido. Ahora, cada componente asume la responsabilidad absoluta de importar explícitamente y de manera exclusiva las dependencias que necesita para operar (por ejemplo, `IonicModule` o `ReactiveFormsModule`). Esta práctica erradica el acoplamiento y evita la carga silenciosa de librerías o servicios innecesarios en la memoria del dispositivo.



2.2. Configuración de Arranque (bootstrapApplication y [app.config.ts](#))

La abolición del módulo raíz exigió una reestructuración profunda del punto de entrada principal de la aplicación (main.ts). El arranque del software ya no depende de la inicialización de un módulo contenedor; en su lugar, se invoca la función nativa bootstrapApplication.

Este método inyecta directamente el componente raíz (AppComponent) acompañado de un archivo de configuración global centralizado denominado app.config.ts. Este archivo estratégico asume el rol de proveer los servicios a nivel de aplicación, encapsulando la inicialización del enrutador (Router), las estrategias de navegación y los proveedores nativos de Ionic. El resultado es un flujo de inicialización declarativo, altamente legible y fácilmente auditable.

2.3. Impacto en el Rendimiento y Modularidad

La migración a Standalone Components genera beneficios tangibles que impactan directamente en la viabilidad técnica del proyecto:

- Optimización del Rendimiento (Performance):

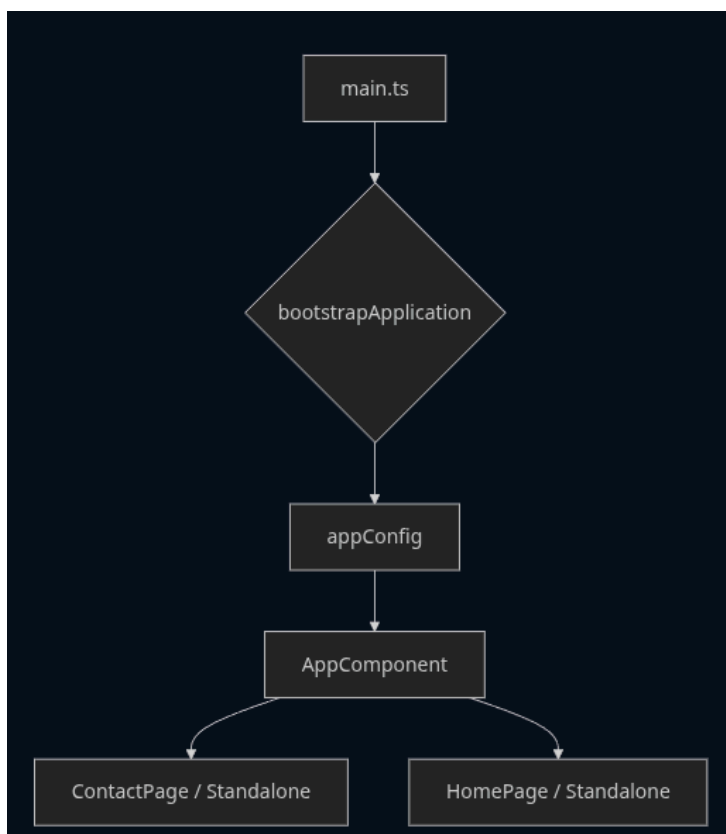
Al contar con componentes independientes, el compilador web (Bundler) puede aplicar procesos de Tree-Shaking (eliminación de código muerto) de forma mucho más agresiva. Esto resulta en la generación de paquetes JavaScript (bundles) considerablemente más reducidos, disminuyendo la latencia de red y mejorando métricas vitales como el First Contentful Paint (FCP), un factor crítico en conexiones móviles inestables.

- Modularidad y Carga Perezosa (Lazy Loading):

La nueva arquitectura simplifica la implementación del Lazy Loading a nivel de enrutador. Al no depender de módulos complejos, la aplicación descarga la lógica y las vistas del "Laboratorio de Consultoría" única y exclusivamente cuando el usuario navega hacia esa sección.

- Mantenibilidad Escalable:

Cada vista se convierte en una unidad funcionalmente autónoma y portable. Esto previene colisiones en la inyección de dependencias y facilita la escalabilidad del sistema, permitiendo que futuros desarrolladores puedan integrar nuevas funcionalidades sin riesgo de corromper la configuración global de la aplicación.



3. INGENIERÍA DE FORMULARIOS REACTIVOS

Para la captura de datos en el "Laboratorio Interactivo de Consultoría", se abandonó el uso de directivas basadas en plantillas (Template-driven forms) como ngModel. En su lugar, se implementó una arquitectura basada en el estado lógico del componente, garantizando un control granular sobre las interacciones del usuario.

3.1. Implementación de ReactiveFormsModule

La reingeniería del módulo de contacto requirió la importación explícita de ReactiveFormsModule dentro de los metadatos del Componente Independiente (Standalone). Esta decisión técnica establece al modelo de TypeScript como la única fuente de la verdad (Single Source of Truth).

Mediante la inyección de la clase FormBuilder, se instanció un objeto FormGroup que agrupa lógicamente los campos de entrada: la categoría del servicio, el correo electrónico y los detalles de la consulta. A diferencia de los formularios tradicionales, donde la vista dicta el comportamiento, en esta arquitectura la vista simplemente se suscribe a los cambios de estado emitidos por el modelo reactivo.

3.2. Validación Atómica y Reglas de Negocio (Required, MinLength, Email)

Para asegurar la integridad de los datos antes de que interactúen con el servicio de Micro-CRUD, se acopló un sistema de validación sincrónica a cada FormControl. Estas reglas de negocio se evalúan de manera atómica con cada pulsación de tecla (entrada de usuario):

Categoría (Validators.required):

Asegura que el usuario seleccione obligatoriamente un área de soporte (Desarrollo Web, Arquitectura Cloud, etc.) para clasificar correctamente el ticket en la bitácora.

Correo Electrónico (Validators.email):

Verifica mediante expresiones regulares nativas de Angular que la cadena de texto posea un formato de dirección de red válido.

Detalles de la Consulta (Validators.minLength):

Exige una longitud mínima de caracteres (ej. 10 caracteres) para evitar consultas vacías o carentes de contexto, promoviendo una descripción útil del requerimiento.

CANAL DE COMUNICACIÓN SD2

 REGISTRAR INTERACCIÓN

CORREO ELECTRÓNICO

Prueba

CATEGORÍA DE SERVICIO

Desarrollo Web

DETALLES DE LA CONSULTA

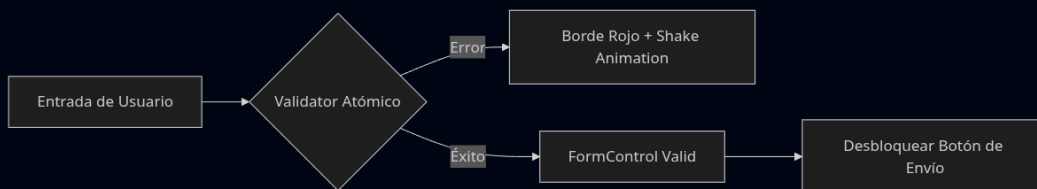
Prueba

 **EJECUTAR ENVÍO**

3.3. Reactividad en la Interfaz (Desbloqueo condicional del botón de envío)

La implementación reactiva permite un enlace directo entre la validez lógica de los datos y la experiencia visual del usuario. Como se ilustra en la arquitectura lógica del componente, el flujo de validación determina el comportamiento del botón de acción principal.

Cuando la entrada del usuario es procesada por el validador atómico, el estado del formulario se actualiza en tiempo real. Si los controles contienen errores, la interfaz bloquea la interacción, evitando el envío prematuro de datos. Por el contrario, cuando la entrada cumple todas las reglas de negocio (éxito), la propiedad `valid` del `FormGroup` retorna un valor afirmativo. Esto desencadena la reactividad en la interfaz (mediante la directiva `[disabled]`), procediendo a desbloquear y habilitar automáticamente el botón de envío para registrar la interacción en la bitácora.



4. GESTIÓN DE ESTADO Y MICRO-CRUD

Para elevar la complejidad técnica de la Sesión Didáctica y superar el alcance de un formulario estático, se incorporó un sistema de persistencia temporal. Esta implementación transforma la captura de datos en un ecosistema interactivo, estructurado bajo patrones de diseño que garantizan la escalabilidad y el mantenimiento del código.

4.1. Arquitectura de Inyección de Dependencias (ContactService)

La separación de responsabilidades (principio de Responsabilidad Única de SOLID) es un estándar innegociable en el desarrollo con Angular. Para aislar la lógica de negocio de la capa de presentación, la arquitectura dicta que el componente de la vista (ContactPage) inyecta un servicio especializado denominado ContactService.

A través del sistema de inyección de dependencias, el componente delega toda la responsabilidad del procesamiento de información, manteniendo su propio código ligero y enfocado exclusivamente en la recolección de eventos de la interfaz y la respuesta visual.

4.2. Operaciones en Memoria Local (Create, Read, Delete)

El ContactService inyectado asume el rol de controlador de datos, siendo responsable de gestionar el "Historial en Memoria". Al carecer de un backend en esta fase del proyecto, se construyó un motor Micro-CRUD soportado por arreglos locales en TypeScript.

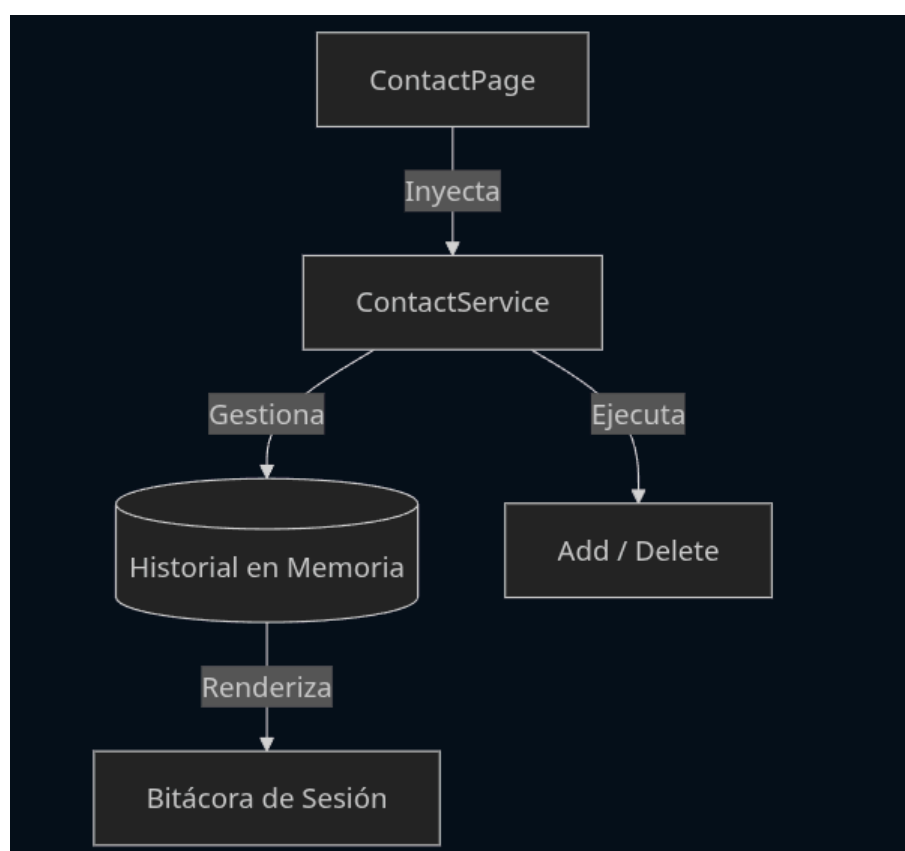
El servicio expone y ejecuta métodos transaccionales específicos para añadir (Add) y eliminar (Delete) iteraciones dentro de este arreglo. Cuando el formulario emite un estado válido, el servicio transforma esos datos en un objeto estructurado (Create) y lo almacena en el estado local. Asimismo, provee la lógica necesaria para purgar registros individuales mediante un identificador único, garantizando un control absoluto sobre el ciclo de vida de los datos temporales.

```
src > app > services > contact.service.ts > ContactService > enviar
19 export class ContactService {
20
21
22
23
24
25
26 /**
27  * @method enviar
28  * @description Registra la interacción localmente y abre WhatsApp.
29  */
30 public enviar(email: string, mensaje: string, categoria: string = 'Soporte Académico'): void {
31     const nuevoMensaje: InteraccionMensaje = {
32         id: Date.now(),
33         email,
34         mensaje,
35         fecha: new Date(),
36         categoria
37     };
38
39     // Almacenamiento en memoria (Micro-CRUD: Create)
40     this._historial.push(nuevoMensaje);
41
42     const textBase =
43         `Hola Juan! 📞\n` +
44         `*Categoria:* ${categoria}\n` +
45         `*📧 Email:* ${email}\n` +
46         `*📄 Mensaje:* ${mensaje}`;
47
48     const url = `https://wa.me/${this.NUMERO_WA}?text=${encodeURIComponent(textBase)}`;
49     window.open(url, '_blank');
50 }
51
52 /**
53  * @method obtenerHistorial
54  * @description Retorna los mensajes (Micro-CRUD: Read)
```

4.3. Renderizado Dinámico de la Bitácora de Sesión

La eficiencia de la gestión de estado se evidencia en su sincronización con la vista. El "Historial en Memoria", una vez gestionado y actualizado por el servicio, renderiza directamente la "Bitácora de Sesión" en la interfaz del usuario.

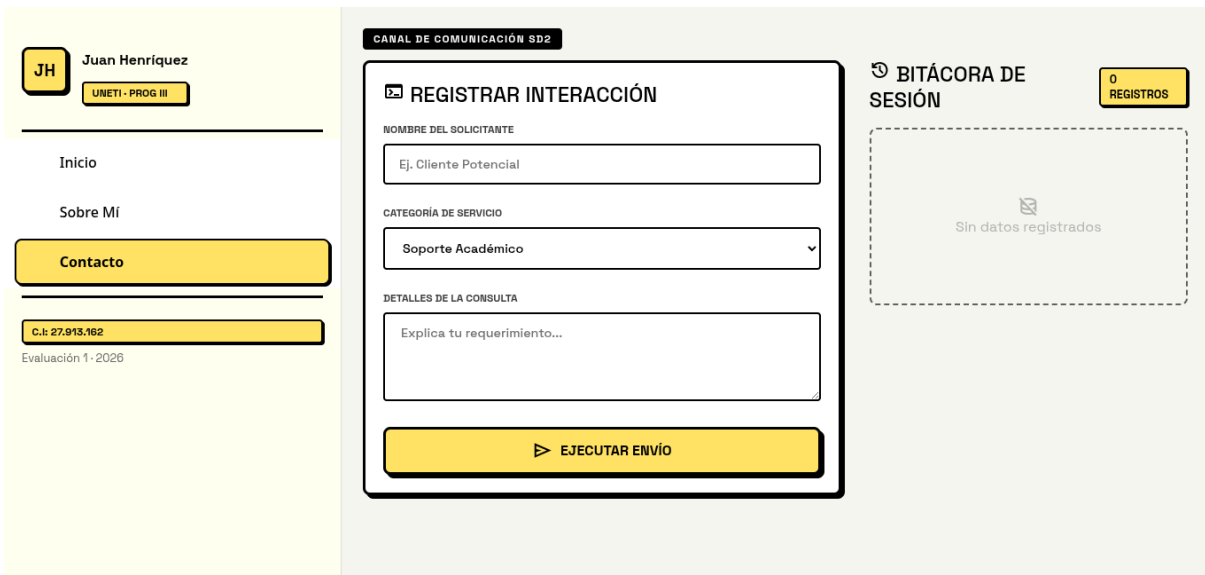
Al aprovechar el motor de reactividad y detección de cambios de Angular, el flujo de datos es unidireccional y automático. Cualquier ejecución de las operaciones Add o Delete altera el estado de la memoria, lo que a su vez dispara una actualización instantánea en el DOM. Esto permite que la Bitácora de Sesión se construye dinámicamente, mostrando u ocultando tickets de soporte en tiempo real y proporcionando un entorno de trabajo fluido y sin interrupciones.



5. DISEÑO UI/UX Y FEEDBACK MULTIMEDIA

5.1. Estructura visual: Patrón Bento-Grid

Para la organización de la interfaz se adoptó el estándar Bento-Grid, una tendencia moderna de diseño que compartimenta la información en bloques o 'tiles' de distintos tamaños. Esta técnica permite jerarquizar el contenido de forma intuitiva: el formulario (acción principal) ocupa el bloque mayor, mientras que la bitácora y el estado de disponibilidad se presentan en bloques secundarios, optimizando el escaneo visual del usuario.



5.2. Feedback Nativo Integrado (ToastController de Ionic)

La comunicación del sistema se gestiona mediante el ToastController de Ionic. Al enviar un mensaje o eliminar un registro, la aplicación dispara notificaciones flotantes asíncronas con códigos de color semánticos (Verde: Éxito / Rojo: Eliminación). Esta implementación garantiza un 'feedback loop' constante sin interrumpir la navegación, elevando la percepción de fluidez.



The screenshot displays the UNETI application interface. On the left, a sidebar menu includes a user profile for Juan Henriquez (UNETI - PROG III) and navigation options: Inicio, Sobre Mí, and Contacto. The main content area is titled 'CANAL DE COMUNICACIÓN 992' and features a 'REGISTRAR INTERACCIÓN' form. This form includes fields for 'CORREO ELECTRÓNICO' (Ej. usuario@unet.edu.ve), 'CATEGORÍA DE SERVICIO' (Soporte Académico), and 'DETALLES DE LA CONSULTA' (Explica tu requerimiento...). A yellow button labeled 'EJECUTAR ENVÍO' is at the bottom of the form. To the right, a 'BITÁCORA DE SESIÓN' section shows a list of two records, both marked as 'PROCESADO'. A green toast notification at the bottom center reads '¡Interacción registrada con éxito!'.

5.3. Micro-interacciones (Badges de estado pulsantes)

Se incorporaron micro-interacciones mediante animaciones CSS (Keyframes). El badge de disponibilidad en el Hero Card cuenta con una animación de pulso infinito (animate-pulse), indicando que el sistema está 'vivo' y listo para recibir entradas. Asimismo, el uso de sonidos (Audio API) al enviar o borrar refuerza la respuesta táctil del sistema, creando una experiencia inmersiva multicanal.



6. CONCLUSIONES

La reingeniería ejecutada durante esta Sesión Didáctica demostró con éxito la viabilidad de modernizar una aplicación híbrida mediante la arquitectura Standalone de Angular.

La eliminación de los NgModules no solo simplificó la estructura del repositorio, sino que optimizó la carga de recursos de la interfaz. Asimismo, la transición hacia Formularios Reactivos y la implementación de un servicio de gestión de estado local (Micro-CRUD) transformaron una vista estática en un "Laboratorio Interactivo de Consultoría" completamente funcional, garantizando la integridad de los datos y proporcionando una experiencia de usuario fluida y con feedback nativo en tiempo real.

REFERENCIAS BIBLIOGRÁFICAS

- **Angular Team. (2024).** *Standalone Components Guide*. Documentación oficial de Angular. Recuperado de: <https://angular.dev/guide/components/importing>
- **Angular Team. (2024).** *Reactive Forms in Angular*. Documentación oficial de Angular. Recuperado de: <https://angular.dev/guide/forms/reactive-forms>
- **Ionic Framework. (2024).** *Toast Controller API*. Documentación oficial de componentes UI. Recuperado de: <https://ionicframework.com/docs/api/toast>
- **Mozilla Developer Network (MDN). (2023).** *CSS Grid Layout* (Base estructural para el diseño Bento-Grid). Recuperado de:
https://developer.mozilla.org/es/docs/Web/CSS/CSS_grid_layout